

Spring IoC Container, Beans & Annotation-Based Configuration — Quick Revision

1. Core Idea

A class should focus only on its own job, not create its own dependencies. `OrderService` should place orders, not create `PaymentService` itself — pass it in from outside (Dependency Injection).

2. First-Principles Understanding

Every app needs: who creates objects, who connects them, who manages their lifecycle. Without Spring, `main()` does this manually. With Spring, the **Spring IoC Container** does it automatically — this is Inversion of Control (IoC).

3. Basic Project Setup

Create a Maven project + add `spring-context` dependency. This gives `ApplicationContext`, annotation config, component scanning, and bean/DI support.

4. What Is a Spring Bean?

A **bean** = an object whose creation, dependency wiring, and lifecycle are managed by the Spring IoC container (not just a plain Java object).

5. Two Configuration Styles

- **Annotation-based** (modern): `@Component`, `@Configuration`, `@ComponentScan`, `@Autowired`, `@Bean`.
- **XML-based** (older/legacy): still seen in old projects, not preferred now.

6. Why `Student.class` Matters (Reflection)

`Student.class` is not an object of `Student` — it's metadata (a `Class` object) describing the class (fields, methods, constructors, annotations). Spring uses this internally to read configuration.

7. Telling Spring Which Classes to Manage

`@Component` on a class tells Spring "this is eligible to become a bean." But Spring also needs to know *where* to look — that's `@ComponentScan`'s job.

8. `ApplicationContext` — The Spring IoC Container

`ApplicationContext` is the interface representing the container — reads config, creates beans, resolves dependencies, manages lifecycle, provides beans on request. Since it's an

interface, we use an implementation like `AnnotationConfigApplicationContext`.

9. `AnnotationConfigApplicationContext`

An implementation of `ApplicationContext` that starts a Spring container using **annotation-based** configuration, reading instructions from a config class (e.g. `AppConfig.class`).

10. `AppConfig.class`

A configuration class marked `@Configuration` + `@ComponentScan("package")` — tells Spring: this is config, scan this package, find `@Component` classes, create & wire their beans.

11. `@Configuration`

Marks a class as a **source of bean definitions** — tells Spring "this class may contain setup instructions" (like `@ComponentScan`, `@Bean` methods).

12. `@ComponentScan`

Tells Spring which package(s) to search for `@Component`-annotated classes. If no package is given, it scans the package of the class it's on (and sub-packages) by default.

13. `getBean()`

Asks the container for a bean: `context.getBean(OrderService.class)` means "give me the `OrderService` bean." Fails if Spring doesn't know about that class (not annotated, wrong package, not registered).

14. Types of DI in Spring

Three types: **Constructor injection**, **Field injection**, **Setter injection**.

15. Constructor Injection

Dependency passed via constructor. If a bean has only one constructor, `@Autowired` on it is optional — Spring auto-detects it.

16. Why Constructor Injection Is Preferred

- Dependency is **mandatory** — object can't exist without it.
- Field can be marked `final` — safer, immutable.
- Class can be tested **without Spring** by just calling `new` manually.

17. Field Injection

`@Autowired` directly on a field. Works via reflection, but **not preferred**: dependency is hidden, hard to test without Spring, can't be `final`, object may exist incomplete before injection.

18. Setter Injection

Spring creates the object first (no-arg constructor), then calls a setter to inject the dependency. Good for **optional** or **changeable** dependencies; constructor injection preferred for mandatory ones.

19–20. What Happens When Spring Starts (Steps 1–5)

1. Spring starts the container (`ApplicationContext`).
2. Reads `AppConfig.class`.
3. Processes `@ComponentScan` to know which package to search.
4. Finds classes marked `@Component`/`@Service`/`@Repository`/`@Controller`.
5. Creates **BeanDefinitions** first — metadata (class, scope, dependencies, creation strategy) about how to build each bean, before actually creating objects.

21. Steps 6–9: Object Creation & Injection

6. Spring creates bean objects for beans with no dependencies first (e.g. `PaymentService`).
7. For beans needing dependencies (e.g. `OrderService`), Spring checks the constructor to see what's needed (**dependency resolution**).
8. If the needed bean exists in the container, Spring injects it during object creation.
9. Once ready, the app can use the bean (`orderService.placeOrder()`).

22. What If No Matching Bean Exists?

Spring throws `No qualifying bean of type '...' available`. Common causes: missing `@Component`, package not in `@ComponentScan`, no `@Bean` method registering it.

23. What If Multiple Matching Beans Exist? (Bean Ambiguity)

If two classes implement the same interface (e.g. `UPIPaymentService` & `CardPaymentService` both implement `PaymentService`), Spring gets confused about which to inject — solved using `@Primary` or `@Qualifier`.

24. `@Primary`

Marks one implementation as the **default choice** when multiple beans match a type.

25. `@Qualifier`

Used to **explicitly pick a specific bean by name** when multiple candidates exist — overrides ambiguity by naming exactly which bean to inject.

26. Custom Bean Names with `@Component`

By default bean name = class name (lowercased). We can override: `@Component("upi")`, `@Component("card")` — then reference these names via `@Qualifier`.

27. `@Qualifier` with Field & Setter Injection

`@Qualifier` can be combined with `@Autowired` on fields or setters too, not just constructors — though constructor injection is still generally preferred.

28. `@Primary` + `@Qualifier` Together

If both are used, `@Qualifier` **wins** — because it's a specific choice, which overrides the default choice given by `@Primary`.

29. Why We Need `@Bean`

`@Component` only works on classes we own. For classes from **external libraries** (can't add annotations to them), we use `@Bean` inside a `@Configuration` class to manually register them.

30. Custom Object Creation with `@Bean`

Also useful when object creation needs custom logic/parameters that Spring can't guess automatically (e.g. a `User(name, email)` constructor with no clear default values).

31. What `@Bean` Means

Used on a method inside a `@Configuration` class. Tells Spring: call this method, take the returned object, register and manage it as a bean.

32. Custom Bean Name with `@Bean`

Default bean name = method name. Can override: `@Bean("myPaymentService")`, then fetch via `context.getBean("myPaymentService")`.

33. `@Bean` with Dependencies

A `@Bean` method can take parameters — Spring resolves and injects the needed bean (e.g. `PaymentService`) automatically when calling that method to build another bean (e.g. `OrderService`).

34. `@Component` vs `@Bean`

Point	<code>@Component</code>	<code>@Bean</code>
Where	On a class	On a method

Point	@Component	@Bean
Style	Automatic detection	Manual registration
Best for	Our own classes	External library classes / custom creation
Needs scanning?	Yes	No (but config class must load)
Bean name	Class name	Method name

35. Avoid Registering Same Bean Twice

Don't use both `@Component` and `@Bean` for the same object unless intentional — causes ambiguity or unexpected bean overriding. Pick one approach per bean.

36. Dependency Resolution with `@Bean`

Same ambiguity rules apply — use `@Primary` or `@Qualifier` inside `@Bean` methods to control which implementation gets injected.

37. Why Not Put Everything in `main()`?

`main()` should only **start** the app. If it also creates/wires all objects, it becomes a manual container again — defeating the purpose of Spring. Keep `main()` simple: start context, get bean, use it.

38. `BeanFactory` vs `ApplicationContext`

`BeanFactory` = basic container interface (foundation). `ApplicationContext` = builds on top of it, adds features (lifecycle management, event publishing, i18n, etc.) — commonly used in real apps.

39. Final Flow Summary

Start container → read `AppConfig` → process `@Configuration` & `@ComponentScan` → scan packages → find `@Component` classes → create `BeanDefinitions` → resolve dependencies → create bean objects → inject dependencies → store beans → return via `getBean()`.

40–41. Key Takeaways (Golden Points)

- Bean = object managed by Spring IoC container.
- `ApplicationContext` = the container; `AnnotationConfigApplicationContext` starts it with annotations.
- `@Configuration` = source of bean definitions; `@ComponentScan` = where to search; `@Component` = eligible to become a bean.

- Constructor injection preferred (mandatory, testable, `final`-safe); field injection discouraged; setter injection for optional deps.
- Spring builds `BeanDefinition` metadata first, then creates actual objects.
- Ambiguity (multiple matching beans) solved via `@Primary` (default) or `@Qualifier` (specific, wins over `@Primary`).
- `@Bean` = manual, method-level bean registration (for external classes or custom creation logic); `@Component` = automatic, class-level.
- Don't duplicate a bean via both `@Component` and `@Bean`.
- **One-liner:** Spring IoC Container creates objects, wires dependencies, manages lifecycle, and gives ready-to-use beans when required.